Re-integrating Prosit (and PTM) Koina Models into Pioneer BuildSpecLib

A phased plan toward phosphoproteomics support

Pioneer.jl — prepared for N. Wamsley

2026-07-02

1. Executive summary

Pioneer historically supported several Koina fragment-intensity models (Prosit, UniSpec, AlphaPept-Deep) alongside Altimeter and Chronologer. That multi-model support was removed on **2026-05-01** (commit `6bab0dadb`, *"Consolidate SearchDIA pipeline"*) to simplify the codebase down to a single fragment model (Altimeter, a `SplineCoefficientModel`) and a single RT model (Chronologer, a `RetentionTimeModel`).

We now want Prosit back — specifically because Koina hosts **Prosit models trained with post-translational modifications (PTMs)**, opening a path to phosphoproteomics. This document is the plan.

What I verified before writing this plan (not assumptions):

- This environment **can reach and run inference on** `koina.wilhelmlab.org`. I ran real requests, not just health checks.
- The PTM intensity model `Prosit_2024_intensity_PTMs_gl` works: I sent `S[UNIMOD:21]PEPTIDEK` (phospho-Ser) and the b2 ion correctly shifted +79.966 Da.
- A **matched iRT model** `Prosit_2024_irt_PTMs_gl` exists and returns iRT for the same PTM peptides.
- **Chronologer already accepts PTMs** — I sent phospho (`UNIMOD:21`), oxidation (`UNIMOD:35`), and carbamidomethyl (`UNIMOD:4`) and it returned shifted RTs. This means for the phospho case we have **two** viable RT paths (reuse Chronologer, or Prosit's matched iRT).
- The old Prosit code and a **working Prosit build config** still exist in git history and on disk (`data/test_build_spec_lib/scenario_prosit/params_prosit.json`, pointing at `minimal_protein.fasta` — a single-protein FASTA ideal for the first test).

The three real engineering problems (all confirmed against the code and the API):

1. **Raw scalar intensities, not splines.** Altimeter returns NCE-dependent B-spline *coefficients*; Prosit returns a single fixed intensity vector. Different storage type, different search-time evaluation.
2. **No NCE calibration.** Altimeter's spline *is* the calibration (evaluated at a per-run NCE at search time). Prosit takes a raw `collision_energies` input and needs the old static-NCE + charge-adjustment machinery restored.

3. **Monoisotopic vs total-abundance fragment intensities.** This is the subtle one. Pioneer's library stores *total* fragment abundance across the isotope envelope and redistributes it at search time by sulfur count (`getFragIsotopes!`). Prosit predicts only the **monoisotopic** peak. Because the monoisotopic fraction of total abundance varies with fragment mass/composition, feeding Prosit's number in directly is *inconsistent across fragments*. We need a defined conversion.

The plan below re-adds Prosit surgically (the removal commit is **not** cleanly revertible — it bundled unrelated refactors), validates against Altimeter/Chronologer on non-PTM data first, and only then moves to phosphoproteomics.

2. Goals, non-goals, and phasing

2.1 Definition of done for Phase 1 (this plan's primary target)

> Pioneer can build a spectral library using a Prosit Koina model for fragment intensities, on unmodified peptides, and a DIA search using that library performs **comparably to** the Altimeter/Chronologer library on the same real data.

"Comparably" = within a small, agreed tolerance on precursor/protein IDs at 1% FDR **and comparable quant** on the **MTAC yeast 3M run** (yeast = fastest full library to build; we set the exact tolerance once we see the reference Altimeter numbers — see §8).

2.2 Phase map

Phase	Outcome	Gate to next phase
0. Scaffolding	Prosit model dispatch re-added; one-protein library builds end to end via Koina.	Library file is produced; fragments look sane vs a hand-checked Koina response.
1. Non-PTM parity	Prosit library searches real DIA data; results benchmarked vs Altimeter.	IDs within tolerance of Altimeter on the shared dataset.
2. PTM plumbing	`Prosit_2024_intensity_PTMs_gl`(+ RT) build phospho libraries; UNIMOD encoding verified.	Phospho library builds; fragment m/z shifts verified for known peptides.
3. Phospho search + localization	Search a phospho DIA dataset; add/verify site localization.	Runs on a benchmark phospho dataset with sane localization.
4. Scale	Handle combinatorial PTM library size (indexing, memory, decoys).	Large phospho library builds and searches within resource budget.

Phases 2–4 are **future work** scoped here only enough to make sure Phase 1 choices don't paint us into a corner. Phase 1 is where the near-term effort goes.

2.3 Non-goals (explicitly out of scope for now)

- Re-adding UniSpec and AlphaPeptDeep. The old dispatch made them cheap to add later, and we will preserve that generality, but we will not test or maintain them now.
- Training or fine-tuning any model. We consume Koina as-is.
- Site-localization algorithm design (Phase 3) beyond noting the integration point.

3. Current architecture (what exists today)

Everything dispatches on a concrete `KoinaModelType` subtype (`src/structs/KoinaStructs/KoinaModelTypes.jl`). Today only two remain:

```
abstract type KoinaModelType end
struct SplineCoefficientModel <: KoinaModelType; name::String end  # Altimeter (fragments)
struct RetentionTimeModel     <: KoinaModelType; name::String end  # Chronologer (RT)
```

The removed ones were `InstrumentSpecificModel` (UniSpec, AlphaPeptDeep) and `InstrumentAgnosticModel` (Prosit). **Prosit was an `InstrumentAgnosticModel`** — it ignores instrument type but does take a collision energy.

Build pipeline (all under `src/Routines/BuildSpecLib/`, entry `src/Routines/BuildSpecLib.jl`):

```
FASTA digest + mods + decoys        chronologer/chronologer_prep.jl
   -> precursor table (koina_sequence, charge, collision_energy, mz, sulfur)
RT prediction (Chronologer/Koina)   chronologer/chronologer_predict.jl -> precursors.arrow
Fragment prediction (Altimeter)     fragments/fragment_predict.jl -> raw_fragments.arrow
   prepare_koina_batch / parse_koina_batch   koina/koina_batch_prep.jl, koina_batch_parse.jl
   filter_fragments!(::SplineCoefficientModel, ctx)  (predict-time filtering)
Decode -> library fragment structs  fragments/fragment_parse.jl (build_detailed_frags_from_raw)
Final indexes + serialize           build/build_poin_lib.jl (buildPionLib)
```

Model config lives in `src/Pioneer.jl`:

```
const MODEL_CONFIGS = Dict(
  "altimeter" => (annotation_type=UniSpecFragAnnotation("y1^1"),
                  model_type=SplineCoefficientModel("altimeter"), instruments=Set([])))
const KOINA_URLS = Dict(
  "chronologer" => ".../v2/models/Chronologer_RT/infer",
  "altimeter"   => ".../v2/models/Altimeter_2024_splines_index/infer")
```

`BuildSpecLib.jl` currently **hardcodes** `prediction_model = "altimeter"` and the config schema no longer accepts `prediction_model/instrument_type` (`utils/check_params.jl:116-118` documents this).

Two representations of a fragment already exist in `src/structs/SpectralLibrary/fragment_types.jl`:

- `SplineCompactFrag{N,T}` — carries an `NTuple{N,T}` of spline coefficients; `getIntensity(::SplineCompact ::SplineType)` evaluates the spline at NCE at **search time**. (Altimeter.)
- `DetailedFrag{T}` — carries a scalar `intensity::Float16`; `getIntensity(::DetailedFrag, ::ConstantType)` returns it directly. **Currently unused by the build — this is the natural target for a raw-intensity Prosit fragment.**

So the scalar-intensity path is *scaffolded but disconnected*. That is the single most important fact for scoping: we are re-wiring an existing shape, not inventing one.

4. Koina capabilities (empirically verified 2026-07-02)

Server is a Triton inference server. Useful endpoints:

- `GET /v2/models/{model}/config` — I/O contract
- `POST /v2/repository/index` — full model list
- `POST /v2/models/{model}/infer` — inference (`{"id","inputs":[{name,shape,datatype,data}]}`)

4.1 Models relevant to us (all confirmed present)

Purpose	Model	Notes
Non-PTM intensity	`Prosit_2020_intensity_HCD` (`_CID`, `_TMT`)	Phase 1 workhorse
PTM intensity	`Prosit_2024_intensity_PTMs_gl`	Phase 2; **extra input** `fragmentation_types`
PTM intensity (newer)	`Prosit_2025_intensity_22PTM`, `_40PTM`	more PTM classes
PTM iRT (matched)	`Prosit_2024_irt_PTMs_gl`, `Prosit_2025_irt_*`	RT alternative to Chronologer
Current RT	`Chronologer_RT`	**also accepts PTMs** (verified)

4.2 I/O contracts (verified)

```
Prosit_2020_intensity_HCD
  IN : peptide_sequences STRING[-1], precursor_charges INT32[1], collision_energies FP32[1]
  OUT: intensities FP32[174], mz FP32[174], annotation STRING[174]   # fixed 174 slots

Prosit_2024_intensity_PTMs_gl        # NOTE the 4th input
  IN : peptide_sequences, precursor_charges, collision_energies, fragmentation_types STRING[1]
  OUT: intensities/mz/annotation [174]

Prosit_2024_irt_PTMs_gl :  IN peptide_sequences  -> OUT irt FP32[1]
Chronologer_RT          :  IN peptide_sequences  -> OUT rt  FP32[1]
```

Contrast — Altimeter is structurally different (this is why it needs its own path and Prosit needs the *other* path):

```
Altimeter_2024_splines   : OUT coefficients FP32[4,200], knots FP32[8]   # NCE-dependent spline
Altimeter_2024_reisotope : IN fragment_masses, precursor_sulfurs, fragment_sulfurs,
                              isotope_isolation_efficiencies[5] -> OUT intensities[1900]  # 380
```

Altimeter models the **full isotope envelope internally** (sulfur-aware, isolation-efficiency aware) and returns *total* abundance as a spline. Prosit returns a single monoisotopic scalar at one CE. That difference is §5.3.

4.3 Verified inference (phospho round-trip)

`Prosit_2024_intensity_PTMs_gl`, charge 2, CE 27, HCD:

- `PEPTIDEK` -> 28 non-zero peaks, base peak y6.
- `S[UNIMOD:21]PEPTIDEK` -> b2 227.103 -> **265.058** (+79.966, phospho on Ser1). Correct.
- Matched `Prosit_2024_irt_PTMs_gl` -> iRT `[4.06, 17.59, 108.12]`.

Intensities are normalized to base peak = 1.0. Slots that don't apply come back `-1` / `0` and must be filtered (there are always 174 slots).

5. The three divergences and how we handle each

5.1 Raw intensities vs spline coefficients

Fact. Prosit returns `intensities` (scalar per fragment); Altimeter returns `coefficients` + `knots`.

Plan. Prosit fragments become `DetailedFrag{Float16}` (scalar) with `ConstantType` intensity dispatch — the *already-present* unused path. No spline knots file (`spline_knots.jls`), no `SplineFragmentLookup`. The build step (`build/build_poin_lib.jl`) gets a `buildPionLib(::InstrumentAgnosti ...)` method that mirrors the spline one but uses the constant-intensity lookup. The old code did exactly this (`process_batch!` vs `process_spline_batch!`), so we have a template.

5.2 NCE calibration

Fact. Altimeter doesn't consume NCE at build time (spline evaluated per-run at search). Prosit needs a `collision_energies` value per precursor. The old static-NCE machinery was removed: `adjustNCE(NCE, default_charge, peptide_charge, charge_facs)` and `CHARGE_ADJUSTMENT_FACTORS = [1, 0.9, 0.85, 0.8, 0.75]`, plus `nce_params.default_charge` and `nce_params.dynamic_nce` config fields.

Plan. Restore that machinery *only on the Prosit branch* of `chronologer_prep.jl` (where `:collision_energy` is populated). Config gains back `nce_params.default_charge` and `nce_params.dynamic_nce` (validated only when `prediction_model` is a Prosit model). Altimeter's branch is untouched. Note UniSpec's per-run m/z->eV calibration (`get_mz_to_ev_interp`) is a *separate* mechanism we are **not** restoring (UniSpec is out of scope).

Open question for §10: which fixed NCE to use by default for our instruments. Prosit intensity is CE-sensitive; a wrong global NCE hurts spectral-contrast scores. We will sweep a small NCE grid on the benchmark in Phase 1 and pick the best, rather than trusting the config default blindly.

5.3 Monoisotopic vs total-abundance intensities (the crux)

Fact. Pioneer stores *total* fragment abundance and, at search time, `getFragIsotopes!` (`src/Routines/SearchDIA/CommonSearchUtils/getFragIsotopes.jl`) splits that total across isotope peaks using `iso_splines` keyed on the fragment's sulfur count. Prosit predicts the **monoisotopic** peak only. The monoisotopic fraction of a fragment's total abundance depends on its mass and composition (more mass -> smaller monoisotopic fraction), so Prosit's raw number is *not* on the same scale across fragments as Pioneer's total-abundance convention.

Two candidate strategies:

- **(A) Convert monoisotopic -> total at build time (recommended).** For each Prosit fragment, divide the predicted monoisotopic intensity by the expected monoisotopic isotope fraction for that fragment's mass and sulfur count (the same `iso_splines` model Pioneer already owns). The stored value is then a *total abundance* consistent with the Altimeter convention, and **the entire search-side isotope path is reused unchanged.** This is the cleanest and keeps one search code path.
- **(B) Store monoisotopic directly, special-case search.** Add a per-library flag that tells `getFragIsotopes!` the stored intensity is already monoisotopic and to build the envelope up from it (multiply by the envelope) rather than redistribute. More search-side surface area, but avoids a build-time division that can amplify noise for high-mass, low-fraction fragments.

Decision (agreed): strategy (A). It saves the search-time work of a special isotope path, keeps the code much simpler, isolates the change to the build side, and leaves the hot search path identical — which also makes the Phase-1 Altimeter comparison cleaner (only the library differs, not the search code). We still validate (A) numerically in Phase 0 by comparing a converted-Prosit fragment's implied envelope against a matched Altimeter/theoretical envelope for a handful of peptides before trusting it at scale, and keep (B) documented only as a fallback if (A) proves numerically fragile for high-mass, low-fraction fragments.

5.3.1 Concrete mechanism (decided)

Same isotope model everywhere. The mono->total conversion uses the *identical* `IsotopeSplineModel` (`src/utils/isotopeSplines.jl:85`) that search uses in `getFragIsotopes!` (`src/Routines/SearchDIA/CommonSe`

```
f0    = iso_splines(min(sulfur,5), 0, frag_mz*frag_charge)   # monoisotopic fraction
total = prosit_mono_intensity / f0                            # stored library intensity
```

This closes exactly: at search the reconstructed monoisotopic peak is `iso_splines(sulfur,0,mass)*total = P`, so Prosit's predicted peak is preserved and the M+1/M+2… are synthesized — the same treatment Altimeter's stored total receives. Using the same splines at build and search is what makes strategy A correct; a mass-only / averagine approximation is explicitly rejected.

Processing order (decided). Per precursor, within each predicted batch: metadata-filter -> **convert mono->total (splines)** -> **rank by total, keep topN** -> write. The convert must precede the rank because f0 is mass-dependent (~1.3-2.9x across the fragment mass range) and reorders fragments.

Where it runs (decided). `raw_fragments.arrow` is written *already topN-filtered* today (`filter_fragments!` runs at `fragment_predict.jl:186`, before the Arrow write at `:143-147`; ~12% of predicted fragments survive). So the Prosit convert+rank+topN must run at **predict time** inside a `filter_fragments!(::InstrumentAgnosticModel)` overload — *not* at decode, or `raw_fragments.arrow` would carry all ~174 fragments/precursor and blow up.

What the predict-time path needs (new plumbing) — reuses existing functions. To evaluate the splines before the write, the filter context (`build_spline_frag_filter_ctx`) gains: (1) the loaded `IsotopeSplineModel` (load the isotope XML once), and (2) per-fragment **sulfur**, computed early from data we already have. The exact functions exist:

- `count_sulfurs!` (`fragment_parse.jl:199`) builds `seq_idx_to_sulfur` (1 per C/M residue – mod sulfur diffs at their positions) from `(plain_sequence, mods_iterator, mods_to_sulfur_diff)`.
- `get_fragment_indices(base_type, index, seq_len)` (`:236`) gives a fragment's residue span (b/a/c -> `(1,index)`; y/x/z -> `(seq_len-index+1, seq_len)`; p -> whole).

`base_type/frag_index/charge` are already on the annotation the filter reads (`fragment_predict.jl:328-350`) and `prec_len` is on the ctx; the only new ctx input is the precursor **sequence + mods**. Plan: precompute a per-precursor **cumulative sulfur prefix array** when building the ctx (it already loops precursors at `:256-259`); then per fragment, `sulfur = prefix[stop] - prefix[start-1]` (O(1)).

Carry forward, don't recompute. For Prosit, `raw_fragments.arrow` stores the already-converted `intensities` (= total) *and* the per-fragment `sulfur` as columns; the decode step packs them straight into `DetailedFrag`. This guarantees identical sulfur/intensity from predict -> decode -> search (search reads `getSulfurCount(frag)` at `getFragIsotopes.jl:63`) with no double computation.

Rank filtering inventory (current, for reference). (1) `filter_fragments!(::SplineCoefficientModel)` = the only live per-precursor topN (order-based for Altimeter, since a spline has no scalar to sort by); (2) `getDetailedFrags/filterFrag` in `build_poin_lib.jl` = vestigial, "no longer consulted" (comment at `:902-910`); (3) the `SimpleFrag` fragment-index build (`build_poin_lib.jl:500-543`, `max_rank_index=8`) = a *separate* top-8 cap for the coarse search index, downstream of the detailed topN. Prosit changes only (1) — intensity-based instead of order-based.

6. Concrete attach points (surgical change list)

Each row is a place that currently has *only* a `SplineCoefficientModel` method and needs a parallel Prosit (`InstrumentAgnosticModel`) method. Old implementations are recoverable from `6bab0dadb^` (= `a33f96466`).

Concern	File : site	Prosit method to add
Model type	`structs/KoinaStructs/KoinaModelTypes.jl`	add `InstrumentAgnosticModel` (and keep `InstrumentSpecificModel` stub for future)
Registry	`src/Pioneer.jl` (`MODEL_CONFIGS`, `KOINA_URLS`)	add `prosit_2020_hcd` (+ PTM entries later) w/ `GenericFragAnnotation`, URL
Request	`koina/koina_batch_prep.jl:21`	`prepare_koina_batch(::InstrumentAgnost` adds `collision_energies` (and `fragmentation_types` for PTM models)
Parse	`koina/koina_batch_parse.jl:40`	`parse_koina_batch(::InstrumentAgnostic` reads scalar `intensities+mz+string annotation`

Concern	File : site	Prosit method to add
Filter + rank	`fragments/fragment_predict.jl:298`	`filter_fragments!(::InstrumentAgnostic` metadata filters, then **mono->total via `iso_splines`, rank by total, topN** (§5.3.1). Ctx gains `IsotopeSplineModel` + per-fragment sulfur (needs precursor sequence on ctx).
Annotation	`fragments/fragment_annotation.jl:232`	`Cjl:232FragAnnotation` parser already exists — reuse
Decode	`fragments/fragment_parse.jl:68`	emit `DetailedFrag` from `intensities` (template: old `process_batch!`)
Build	`build/build_poin_lib.jl:132,366`	`buildPionLib/getDetailedFrags(::Instrum` w/ `ConstantType`, no knots
NCE	`chronologer/chronologer_prep.jl:36`	`restore 431536 adjustNCE+CHARGE_ADJUSTMENT_FACTORS` on Prosit branch
Isotopes	predict-time filter (§5.3.1)	mono->total using the **same** `iso_splines` search uses; `total = P / iso_splines(min(sulfur,5),0,mass);` search path unchanged
Config schema	`utils/check_params.jl:116`	re-add `prediction_model`, `instrument_type`, `nce_params.default_charge`
Wiring	`BuildSpecLib.jl:159,279`	branch on model type; skip knot metadata for Prosit

Note the removal commit `6bab0dadb` deleted 2,584 lines across 20 files but *interleaved* unrelated work (`diann_decoys`, a new `koina_client`, mods refactor). **Do not revert it.** Use `git show 6bab0dadb^:<path>` as a *reference*, then hand-port onto today's code.

7. Implementation plan (Phase 0 -> 1), step by step

Each step states its verification, per CLAUDE.md §4.

```
0.1  Re-add InstrumentAgnosticModel + registry entry (prosit_2020_hcd)
     verify: `Pioneer.MODEL_CONFIGS["prosit_2020_hcd"]` resolves; package precompiles.
0.2  prepare_koina_batch + parse_koina_batch for Prosit
     verify: unit test builds a request JSON matching the verified contract (§4.2);
             parse of a captured real response yields intensities+mz+annotation.
0.3  filter + decode -> DetailedFrag; buildPionLib(::InstrumentAgnosticModel)
     verify: one-protein build (minimal_protein.fasta) produces a library file with
             non-empty fragments; spot-check 2-3 fragments' m/z vs a live Koina call.
0.4  Restore NCE adjustment on the Prosit branch
```

```
     verify: collision_energy column varies with charge per CHARGE_ADJUSTMENT_FACTORS.
0.5  Monoisotopic->total conversion (§5.3-A)
     verify: for a few peptides, converted envelope ~matches theoretical/Altimeter
             within tolerance (dedicated numeric test).
1.1  Build a full YEAST library with prosit_2020_hcd (+ Chronologer RT)
     verify: build completes; library size + fragment counts sane.
1.2  DIA search with the Prosit yeast library on the MTAC yeast 3M run (raw from RIS)
     verify: search completes; produce IDs @1% FDR + quant.
1.3  Benchmark vs Altimeter/Chronologer yeast library on the same run
     verify: IDs + quant within agreed tolerance; write up in FINDINGS.md.
1.4  Small NCE sweep; pick default
     verify: chosen NCE documented with the ID curve.
```

We keep the **Altimeter path as the reference** throughout — same search code, same raw files, only the library differs.

7.1 Testing infrastructure to reuse

- One-protein fixture: `data/test_fasta_files/minimal_protein.fasta` (already referenced by the historical `params_prosit.json`).
- The Koina client abstraction (`koina/koina_client.jl`) provides a pluggable client selected per-task via `with_koina_client` (`:84-110`). Four clients exist: `HttpKoinaClient` (production), `SyntheticKoinaClient` (deterministic offline fake — **Altimeter/Chronologer only**, throws on other endpoints, `:265-280`), `RecordingKoinaClient` (captures live `(request,url,response)` triples, `:194-224`), and `FixtureKoinaClient` (replays them, keyed by a canonicalized request+URL hash, errors on cache miss, `:235-261`). A recorded fixture already ships for the existing tests (`test/Routines/BuildSpecLib/koina_fixtures.json`).
- **How we use it for Prosit — with limits.** The record->replay path (`RecordingKoinaClient` -> `save_koina_fixtures` -> `FixtureKoinaClient`) is endpoint-agnostic, so it works for Prosit unmodified: record real Prosit responses once, then iterate on the *downstream* code (parse -> mono->total conversion -> decode -> build) against the fixture without re-hitting Koina. Caveats that bound this: (1) fixtures are keyed on the **exact** request payload, so changing the digest / charges / NCE / batch composition is a cache miss — replay speeds up downstream-code iteration with the request set held fixed, it is not a general "free rebuild". (2) `SyntheticKoinaClient` does **not** cover Prosit as-is (Altimeter/ Chronologer only); for a network-free deterministic *unit* test we either use the fixture path or extend the synthetic client. (3) A full-yeast fixture would be large (the current small-scenario fixture is already ~17.5 MB; a full yeast library would be far bigger) — a local dev artifact, not something to commit. For CI we record a **small** Prosit scenario (one/few proteins) and commit that, keeping the live server a dev-only dependency.

8. Benchmark / real-data validation strategy

We validate at **two tiers**, cheapest first, so we catch problems before paying for a full search.

Tier 1 — library-level diff (fast, no search, no raw file). Build a single-protein library (`minimal_protein.fasta`) with `prosit_2020_hcd` and inspect the library itself: fragment m/z vs a live Koina call, intensity ordering, the monoisotopic->total conversion (§5.3-A) against a

theoretical/Altimeter envelope for a few peptides. This is the Phase-0 gate and needs nothing but the Koina server.

Mirror-plot sanity check (`dev_docs/prosit_ptm_integration/mirror_prosit_vs_altimeter.jl`, built in Phase 0.2): overlays Prosit (up) vs Altimeter (down, spline evaluated at NCE) predicted spectra on a shared m/z axis, swept over an NCE grid, each self-normalized. First run (peptides LGGNEQVTR / GISNEGQNASIK / TASEFDSAIAQDK / LFLQFGAQGSPFLK, NCE {22,26,30}): fragment m/z **align** between the two models (core check passes), Altimeter fragments more with rising NCE, and the **NCE scales of the two models differ** (Altimeter looks under-fragmented at NCE 22 vs Prosit; base peaks don't always coincide) — an early, concrete confirmation that Phase-1 needs the NCE calibration/sweep (§5.2).

Tier 2 — full-pipeline parity (IDs + quant), kept deliberately simple. We need a real DIA run, but the *fastest full library to build* is **yeast** (small proteome), so the Phase-1 benchmark is the **MTAC yeast 3M run**. Even a full yeast Prosit prediction takes some time, but it is far cheaper than human/HeLa. Yeast MTAC raw data comes from **RIS** when we reach this step (not needed until Phase 1.2). Procedure:

1. Build a yeast library with Altimeter/Chronologer (reference) and one with `prosit_2020_hcd` (+ Chronologer RT).
2. Search the same MTAC yeast 3M raw file(s) with each, identical search params.
3. Compare **precursor and protein IDs at 1% FDR** *and* **quant** (e.g. quantified precursor/protein counts and CV/abundance agreement), Prosit vs Altimeter.

Metric detail: we also eyeball spectral-contrast/score distributions, not just counts, because a model swap can shift score scale without changing rank much. We agree the pass tolerance before running (e.g. "Prosit within X% of Altimeter precursor IDs and comparable quant CVs") once we see the reference Altimeter numbers on this run.

9. Phosphoproteomics roadmap (Phases 2-4, scoped, not committed)

Enough detail to avoid Phase-1 dead-ends:

- **Model:** `Prosit_2024_intensity_PTMs_gl` for fragments. The PTM intensity model needs the extra `fragmentation_types` input, so the request-prep method must be PTM-aware.
- **RT (test both):** implement and compare **Chronologer** and **Prosit matched iRT** (`Prosit_2024_irt_PTMs_gl`). Chronologer is generally the more accurate RT predictor, so the default preference is Chronologer *when the requested PTM subset is within Chronologer's supported modification alphabet* (verified to include phospho/ox/ carbamidomethyl; the full alphabet must be enumerated in Phase 2). Fall back to Prosit iRT for PTMs Chronologer cannot represent. We build the plumbing for both regardless.
- **UNIMOD encoding:** Pioneer's `getKoinaSeqs` already emits `[MODNAME]` after residues; the historical format matches ProForma/UNIMOD. Verify the exact token Prosit expects (`[UNIMOD:21]`) matches what `getKoinaSeqs` emits — likely a small naming shim.
- **Site localization (Phase 3):** new algorithmic work. Integration point is post-search scoring; we will need a benchmark phospho dataset (e.g. a synthetic-phosphopeptide or well-characterized enriched dataset) to measure localization accuracy. Flagged as its own design doc when we get there.
- **Library scale (Phase 4):** variable phospho on S/T/Y is combinatorially explosive. Concerns: precursor-table size, fragment-index memory, decoy generation cost, and Koina request vol-

ume (batch of 1000, many batches). We will need to measure and probably cap site combinations per peptide. This is the biggest unknown and deserves its own plan.

10. Risks and open questions

1. **Isotope conversion (§5.3).** Strategy A vs B — the decision I most want your call on. Risk: build-time division amplifies error for high-mass low-fraction fragments.
2. **Default NCE (§5.2).** Wrong global NCE silently degrades Prosit spectral scores. Mitigated by the Phase-1 sweep.
3. **Non-clean removal commit.** Hand-porting from `6bab0dadb^` risks missing a subtle coupling (e.g. the mods refactor changed since). Mitigated by building the one-protein library early (0.3) and diffing fragment output against live Koina.
4. **Prosit 174-slot ceiling.** Prosit caps peptide length / fragment count; very long peptides lose fragments. Altimeter uses 380/1900 slots. For typical tryptic peptides this is fine; flag if the benchmark has long peptides.
5. **CI dependence on Koina.** Mitigated by Recording/Fixture clients (§7.1).

Decisions (resolved 2026-07-02):

- (Q1) Isotope: **strategy A** — convert monoisotopic->total at build (simpler, cheaper). §5.3.
- (Q2) Benchmark: **two tiers** — single-protein library diff, then the **MTAC yeast 3M run** (fastest full library to build; data from RIS at Phase 1.2). Pass tolerance set once we see the reference Altimeter numbers. §8.
- (Q3) Phospho RT: **implement and test both** Chronologer and Prosit iRT; prefer Chronologer when its modification alphabet covers the requested PTMs. §9.
- (Q4) **Agreed** — start on `prosit_2020_hcd`, do not touch PTMs until Prosit build + apply is proven correct on non-PTM data.

11. Documentation & tracking plan

Per your instruction to keep CLAUDE.md and plan files current "as we go":

- This directory `dev_docs/prosit_ptm_integration/` holds the living docs:
  - `PLAN.md` / `PLAN.pdf` — this document.
  - `koina_findings.md` — the verified API/inference reference (contracts, model list).
  - `FINDINGS.md` — created in Phase 1 to record benchmark results and decisions (kept updated each phase).
- CLAUDE.md gets a short pointer to this directory once you approve the approach (I did **not** edit CLAUDE.md yet — that would be premature before sign-off).
- Each phase's completion is recorded in `FINDINGS.md` with the concrete verification numbers, so the plan and reality stay in sync.

12. Key references

- **Removal commit:** `6bab0dadb` (2026-05-01). Reference the pre-removal tree via `6bab0dadb^` (= `a33f96466`, 2026-04-23). Cleanest documented multi-model version: `2e289536e` (2025-08-29).

- **Working historical Prosit config on disk:** `data/test_build_spec_lib/scenario_prosit/params_prosit` (uses `minimal_protein.fasta`, `prediction_model="prosit_2020_hcd"`, `max_frag_rank=50`).
- **Current code:** `src/Routines/BuildSpecLib.jl`, `src/Routines/BuildSpecLib/{koina,fragments,chronc` `src/structs/{KoinaStructs,SpectralLibrary}/`, `src/Pioneer.jl` (`MODEL_CONFIGS/KOINA_URLS`).
- **Verified Koina models:** `Prosit_2020_intensity_HCD`, `Prosit_2024_intensity_PTMs_gl`, `Prosit_2024_irt_PTMs_gl`, `Chronologer_RT` (PTM-capable). Full list in `koina_findings.md`.